

ECHORA

Implementation Workflow and Technology Summary

ECHORA is a real-time sensory-substitution system designed to help blind and visually impaired users navigate, read text, identify people and banknotes, and reach nearby objects.

Document purpose

This document summarizes the implemented architecture, execution flow, feature-level technologies, and feedback mechanisms. It is written as a ready-to-use implementation section for project documentation.

System at a glance

Feature	Primary technologies	Result
Navigation	YOLOv8, stereo depth, ByteTrack	Obstacle distance, direction, safety alerts
Text reading	PaddleOCR PP-OCRv4, OpenCV	Spoken Arabic/English text
Face identification	InsightFace, ArcFace, SQLite	Known or unknown person announcement
Banknotes	Custom YOLOv8 model	Spoken Egyptian Pound denomination
Object reaching	MediaPipe Hands, depth, haptic grid	Directional tactile guidance

1. Overall architecture and control flow

The application is a Python command-line program. `main.py` starts the system and creates a central `ControlUnit`. The `ControlUnit` initializes the camera, AI models, database, audio, haptic services, and state machine, then executes a continuous frame loop.

Per-frame execution sequence

- The OAK-D camera produces synchronized RGB, stereo-depth, and IMU data.
- The obstacle detector analyzes the RGB frame and combines detections with depth measurements.
- Background probes check for text, faces, banknotes, and interactable objects.
- The state machine selects the appropriate operating mode in automatic operation.
- The active mode generates speech, spatial sound, haptic feedback, or a database update.
- The loop repeats continuously while timing metrics and logs are collected.

Performance approach

Expensive OCR and face-identification operations execute in background threads. OCR calls are protected by a lock because its inference engine is not thread-safe. Detection results are cached where possible, and models select CUDA, Apple MPS, or CPU based on available hardware.

Automatic state transitions

Feature	Primary technologies	Result
Navigation	Default mode	Continuously detects and announces hazards
OCR	Text within 2 m plus low motion	Reads stable detected text
Interaction	Interactable object within 0.8 m	Guides the user's hand to the object
Face ID	Repeated high-confidence face detection	Identifies a registered person
Banknote	Repeated banknote detection plus low motion	Classifies the denomination

Transitions require consecutive confirming frames and minimum dwell times. This prevents noisy single-frame detections from causing unwanted mode switches. A danger obstacle has higher priority and can force a return to Navigation mode.

2. Camera and sensor acquisition

Technologies: DepthAI, Luxonis OAK-D, OpenCV, NumPy.

The camera module configures an OAK-D pipeline with a 1280 x 800 RGB camera, left and right mono cameras, stereo-depth processing, and an IMU. StereoDepth aligns depth with the RGB stream, allowing each detected object to be assigned a distance in millimetres. A dedicated IMU thread stores current acceleration and gyroscope data; acceleration magnitude is used to estimate motion and suppress still-image features while the user is moving.

3. Navigation and obstacle awareness

Technologies: Ultralytics YOLOv8, PyTorch, OpenCV, stereo depth, ByteTrack, optional Ollama/Gemma VLM.

Feature flow

- The RGB frame is analyzed by a custom accessibility YOLOv8 model and, when enabled, a second COCO YOLOv8 model.
- Only relevant classes such as people, furniture, doors, stairs, vehicles, bottles, phones, and switches are kept.
- For each bounding box, the detector samples the depth map, calculates distance, and derives a horizontal viewing angle.
- ByteTrack-style tracking stabilizes identities across frames and supports approach-speed estimates.
- Objects are categorized as Danger (under 1.5 m), Warning (1.5 to 3 m), Safe, or Unknown if depth is unavailable.

Feedback behavior

- The closest danger object receives urgent spoken feedback, a spatialized sound, and a danger haptic pulse.
- Warning objects receive speech and spatial sound, with less urgency.
- When no obstacle is present, the system announces that the path is clear.
- When navigation is clear, an optional Gemma model through Ollama can create a short scene description for context.

4. OCR text reading

Technologies: PaddleOCR PP-OCRv4, PaddlePaddle, OpenCV, optional CUDA acceleration.

- A background probe estimates whether readable text is present and measures the nearest text region with the depth map.
- OCR mode starts when text is sufficiently close and the user is stable.
- Preprocessing includes brightness-aware CLAHE, gamma correction for overexposure, Hough-line skew correction, unsharp masking, and bilateral denoising.
- PaddleOCR detects and recognizes text. It supports English, Arabic, or both according to configuration.
- Low-confidence, very small, or non-text-like detections are rejected. Arabic and English detections are overlap-deduplicated.
- Stable, cleaned text is spoken. After five seconds with no reading, the user is asked to adjust distance or camera angle.

5. Face identification and registration

Technologies: InsightFace buffalo_sc, RetinaFace, ArcFace 512-dimensional embeddings, cosine similarity, SQLAlchemy, SQLite.

Recognition flow

- A low-resolution face probe runs during navigation to reduce compute cost.
- After stable detection, full-frame InsightFace processing detects the strongest face and produces an ArcFace embedding.
- The embedding is compared with stored face embeddings using cosine similarity.
- A match needs both a minimum similarity score and a margin above the second-best candidate.
- The result must remain stable across multiple frames before speech output.
- For a known person, the database updates the last-seen time, meeting count, and event log. Otherwise the user hears that the person is unknown.

Registration flow

- The user presses 6, types a name in the camera window, and confirms it with Enter.
- The system captures a short burst of live frames over approximately two seconds.
- Only clear, large, high-confidence face samples are retained.
- The best normalized embeddings are averaged and saved in SQLite.
- The recognition memory is refreshed immediately, so the newly registered face is available in the current session.

6. Egyptian banknote recognition

Technologies: custom YOLOv8 classification/detection model, PyTorch, temporal majority voting.

- A periodic detector checks whether a banknote is visible.
- When a note is repeatedly visible and user motion is sufficiently low, the application enters Banknote mode.
- The custom YOLOv8 model detects and classifies the note denomination above a confidence threshold.
- A short history buffer confirms stable classifications before speech output.
- Each denomination is announced once, then detection pauses while speech completes and a cooldown expires.

7. Hand-to-object interaction guidance

Technologies: MediaPipe Hands, YOLOv8 detection reuse, stereo depth, NumPy geometry, HTTP requests.

- Navigation detections are filtered for interactable objects such as cups, bottles, phones, remotes, switches, and door handles.
- In Interaction mode, MediaPipe Hands provides the dominant hand's index-fingertip location.
- The application calculates horizontal, vertical, and depth offsets from the fingertip to the target-object center.
- A small interaction state machine chooses IDLE, GUIDANCE, EDGE, or SUCCESS.
- The direction is encoded as a 4 x 5 haptic grid, converted to a 20-bit pattern, and sent asynchronously to a local HTTP endpoint.
- Once the hand remains within the success threshold for the required dwell time, the wristband pulses and the system says 'Object reached.'

8. Audio, haptics, storage, and reliability

Audio feedback

Audio uses pytsx3, Windows SAPI where available, and pygame. Speech is queued by priority so urgent obstacle warnings can be heard before lower-priority notifications. Requests expire after a time-to-live period to prevent stale speech. Pygame supplies danger, warning, and chime sounds, and object angle is converted into left/right stereo panning.

Haptic feedback

The interaction feature sends 20-bit patterns to a local HTTP bridge at /api/pattern. If the bridge is unavailable, the module falls back to a log-only stub. The project also includes a separate generic ESP32 haptic abstraction with stub, serial, BLE placeholder, and Wi-Fi UDP options; its current default is stub mode.

Persistent storage

SQLAlchemy manages the local SQLite database. The database stores people, face embeddings, registration metadata, last-seen information, preferences, and event logs. Face embeddings are stored as 512-dimensional float32 vectors compatible with the InsightFace model.

Configuration and operational controls

Central configuration defines model paths, confidence thresholds, camera resolution, safety distances, OCR languages, state dwell times, audio settings, and hardware constants. Pydantic Settings loads values from a .env file. The command line supports automatic or manual operation, headless execution, audio disabling, debug logging, session logs, OCR language selection, and face-tolerance adjustment.

Implementation conclusion

ECHORA follows a modular, real-time pipeline: capture the user's environment, interpret it with specialized AI models, select the most useful task through a safety-first state machine, and return the result through accessible audio and tactile channels. This design keeps the system extensible while prioritizing timely, understandable feedback for real-world use.